



OPERATING SYSTEM DESIGN

S Thenmozhi

Department of Computer Applications

OPERATING SYSTEM DESIGN

Storage Management & System Security

S Thenmozhi

Department of Computer Applications

Access Matrix

Protection is controlling the access to specific users on specific objects

It can be viewed abstractly as a matrix, called an **Access Matrix**. The rows of the access matrix represent domains, and the columns represent objects. Each entry in the matrix consists of a set of access rights.

OPERATING SYSTEMS DESIGN

Access Control

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

- If a process in Domain D_i tries to do “op” on object O_j , then “op” must be in the access matrix
- User who creates object can define access column for that object
- Can be expanded to dynamic protection
 - Operations to add, delete access rights
 - Special access rights:
 - *owner of O_i*
 - *copy op from O_i to O_j (denoted by “*”)*
 - *control – D_i can modify D_j access rights*
 - *transfer – switch from domain D_i to D_j*
 - *Copy and Owner* applicable to an object
 - *Control* applicable to domain object

Access Matrix with Domains as Objects

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Access Matrix with Copy Rights

domain \ object	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

domain \ object	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

Access Matrix With Owner Rights

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Access Matrix with Control

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Implementation of Access Matrix

- Generally, a sparse matrix
- **Option 1 – Global table**
 - Store ordered triples **<domain, object, rights-set>** in table
 - A requested operation M on object O_j within domain D_i -> search table for $\langle D_i, O_j, R_k \rangle$
 - with $M \in R_k$
 - But table could be large -> won't fit in main memory
 - Difficult to group objects (consider an object that all domains can read)

- Option 2 – Access lists for objects

- Each column implemented as an access list for one object
- Resulting per-object list consists of ordered pairs **<domain, rights-set>** defining all domains with non-empty set of access rights for the object
- Easily extended to contain default set -> If $M \in$ default set, also allow access

- Each column = Access-control list for one object
Defines who can perform what operation

Domain 1 = Read, Write

Domain 2 = Read

Domain 3 = Read

- **Option 3 – Capability list for domains**

- Instead of object-based, list is domain based
- **Capability list** for domain is list of objects together with operations allowed on them
- Object represented by its name or address, called a **capability**
- Execute operation M on object O_j , process requests operation and specifies capability as parameter
- Each Row = Capability List (like a key)

For each domain, what operations allowed on what objects

Object F1 – Read

Object F4 – Read, Write, Execute

Object F5 – Read, Write, Delete, Copy

- **Option 4 – Lock-key**
 - Compromise between access lists and capability lists
 - Each object has list of unique bit patterns, called **locks**
 - Each domain has list of unique bit patterns called **keys**
 - Process in a domain can only access object if domain has key that matches one of the locks

When to adapt What?

- Global table is simple, but can be large
- Access lists correspond to needs of users
 - Every access to an object must be checked
 - Many objects and access rights -> slow
- Capability lists useful for localizing information for a given process
 - But revocation capabilities can be inefficient
- Lock-key effective and flexible, keys can be passed freely from domain to domain, easy revocation



THANK YOU

S Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393